Flash / Metadata逐byte位址與CAN封包移植對照

1. 適用角色與閱讀方式

本節對照 **F13 (MCU CAN device)** 與 **F24 (MCU CAN device)** 的 App metadata、Flash 位址與 CAN 封包。 兩者各自有獨立的 Boot / App 工程，共用欄位格式與更新流程概念。

映像識別碼為 ASCII `CAN\0`（數值 `0x004E4143`，bytes `43 41 4E 00`）。 這是檔案格式的 magic，與 CAN0 / CAN1 等通道編號無關。 CAN channel 與 pin 由各 MCU 工程設定，bridge port 由 Tool 選擇。

Flash絕對位址、BIN offset與metadata offset

```
metadata_base = app_start + 0x100
field_flash_address = app_start + 0x100 + metadata_offset
field_bin_offset = 0x100 + metadata_offset
crc_flash_address = app_start + image_size - 4
crc_bin_offset = image_size - 4
```

例如兩個 device 的 target_id 都位於 metadata offset `0x0C..0x0D`，Flash 位址為 `0x0210C..0x0210D`，BIN offset 為 `0x10C..0x10D`。 多 byte 整數採 little-endian，低位 byte 放在低位址。

區域	F13 (MCU CAN device)	F24 (MCU CAN device)
Code Flash	0x00000..0x1FFFF	0x00000..0x3FFFF
Boot	0x00000..0x01FFF	0x00000..0x01FFF
App 可用範圍	0x02000..0x1FFFF	0x02000..0x3FBFF
App 固定跳轉表	0x02000..0x020FF	0x02000..0x020FF
Metadata 整區	0x02100..0x021FF	0x02100..0x021FF
已定義 core / 含 UPD1	0x02100..0x0217F	0x02100..0x0217F
Reserved FF	0x02180..0x021FF	0x02180..0x021FF
Linked code 起點	0x02200	0x02200
額外保留	debug trace RAM	Flash 0x3FC00..0x3FFFF
Image CRC32	實際 BIN 最後4 bytes	實際 BIN 最後4 bytes

CRC 類型、計算範圍與用途

名稱	存放 / 傳送位置	計算範圍
Image CRC32	BIN最後4 bytes / LE32	BIN[0..image_size-5]，含跳轉表、metadata、reserved及填洞FF；排除CRC自身
Metadata staging CRC32	CAN SEAL或Host BEGIN的command payload	完整metadata 256 bytes；不是多佔用一組metadata內的CRC欄位
CAN block CRC32	BLOCK_BEGIN及BLOCK_COMMIT的Byte4..7	本區塊實際image bytes，最多64；不含control header或未傳送padding
Custom Transfer CRC8	固定8-byte payload的Byte7	該Transfer封包Byte0..6；不是CAN Upgrade的固定Byte7規則
CAN控制器鏈路CRC	CAN frame硬體協定欄位	由CAN控制器處理，不佔本文件Byte0..7的payload
Host Link CRC16	UART frame的header CRC / payload CRC	CRC16/CCITT-FALSE；header覆蓋Byte2..9，payload CRC只覆蓋UART payload，不是image CRC
Intel HEX / S-record checksum	HEX / MOT每筆文字record	燒錄檔record完整性；不是App image CRC32

Image / metadata / block採CRC32/ISO-HDLC，reflected polynomial0xEDB88320，init及xorout0xFFFFFFFF。 CRC提供完整性，不是簽章、認證或防回滾。image_size隨linker結果改變，所以CRC實際address必須重新計算。

2. 每個metadata byte：address、範例值與來源

下表所有offset均相對metadata_base；address / offset以0x標示，value欄是兩位十六進位byte，bytes欄長度為十進位。「產生端來源」對照下一節的範例程式連結；欄位後[0] / [1]等表示該欄位內的byte。 產生來源為各 App 的 `ftable.inc`；F13 由 `ftable_RL78_F13.asm` 組譯，F24 由 `ftable_RL78_F24.asm` 組譯。 兩者的 App 身分由各自的 C header 提供，固定欄位來自各自 Boot 契約。

MCU CAN device：Metadata 欄位與產生端

Metadata offset	bytes	欄位	產生端來源符號 / 位置
0x00..0x03	4	image_magic (ASCII CAN\0)	IMAGE_MANIFEST_MAGIC
0x04..0x04	1	image_format	IMAGE_MANIFEST_FORMAT_INFO
0x05..0x05	1	core_length	IMAGE_MANIFEST_FORMAT_INFO
0x06..0x07	2	flags	IMAGE_MANIFEST_FORMAT_INFO
0x08..0x09	2	protocol_id	IMAGE_MANIFEST_PROTOCOL
0x0A..0x0A	1	protocol_version	IMAGE_MANIFEST_PROTOCOL
0x0B..0x0B	1	checksum_type	IMAGE_MANIFEST_PROTOCOL
0x0C..0x0D	2	target_id	IMAGE_MANIFEST_TARGET
0x0E..0x0E	1	can_mode_mask	IMAGE_MANIFEST_TARGET
0x0F..0x0F	1	can_id_format	IMAGE_MANIFEST_TARGET
0x10..0x13	4	app_version	APP_IMAGE_VERSION
0x14..0x17	4	app_build	APP_IMAGE_BUILD
0x18..0x1B	4	image_size	IMAGE_SIZE_POST_BUILD / post-build
0x1C..0x1F	4	app_start	FLASH_TABLE
0x20..0x23	4	app_capacity	APP_REGION_SIZE
0x24..0x27	4	flash_size	CODE_FLASH_SIZE
0x28..0x2B	4	boot_size	BOOT_RESERVED_SIZE
0x2C..0x2F	4	erase_size	FLASH_ERASE_BLOCK_SIZE
0x30..0x33	4	nominal_rate_mask	CAN_NOMINAL_BITRATE_MASK
0x34..0x37	4	data_rate_mask	CAN_DATA_BITRATE_MASK
0x38..0x3B	4	request_id	CAN_REQUEST_ID
0x3C..0x3F	4	response_id	CAN_RESPONSE_ID
0x40..0x4F	16	mcu_part ASCII	MCU_PART_0..1 / zero padding
0x50..0x5B	12	mcu_family ASCII	MCU_FAMILY_0..1 / zero padding

Metadata offset	bytes	欄位	產生端來源符號 / 位置
0x5C..0x5F	4	minimum_size	MINIMUM_IMAGE_SIZE
0x60..0x63	4	UPD1 magic	UPD1 policy
0x64..0x64	1	policy_version	UPD1 policy
0x65..0x65	1	policy_size	UPD1 policy
0x66..0x66	1	image_role	UPD1 policy
0x67..0x67	1	update_transports	UPD1 policy
0x68..0x6B	4	irq_abi	UPD1 policy
0x6C..0x6F	4	minimum_boot	UPD1 policy
0x70..0x73	4	device_uart_baud	UPD1 policy
0x74..0x77	4	version_control	APP_IMAGE_VERSION_CONTROL
0x78..0x7F	8	reserved_zero	UPD1 policy
0x80..0xFF	128	reserved_FF	reserved region

F13 的來源符號由 ftable.inc 產生；F24 的相同欄位由 device_common.ps1 的 Get-DeviceMetadata 依 App C header / Boot config 產生。

逐 byte 實體位址與範例值

方括號是欄位內byte索引；LE數值的[0]是最低有效byte。ASCII依字元順序，reserved依原始順序。 範例值取配套BIN，不是所有專案的固定值；每個metadata byte皆各列一行，包含128個reserved FF。

Metadata 0x00..0x1F

offset	F13 device address	value	F13欄位[byte]	F24 device address	value	F24欄位[byte]
0x00	0x02100	43	image_magic (ASCII CAN\0)[0]	0x02100	43	image_magic (ASCII CAN\0)[0]
0x01	0x02101	41	image_magic (ASCII CAN\0)[1]	0x02101	41	image_magic (ASCII CAN\0)[1]
0x02	0x02102	4E	image_magic (ASCII CAN\0)[2]	0x02102	4E	image_magic (ASCII CAN\0)[2]
0x03	0x02103	00	image_magic (ASCII CAN\0)[3]	0x02103	00	image_magic (ASCII CAN\0)[3]
0x04	0x02104	06	image_format[0]	0x02104	06	image_format[0]
0x05	0x02105	80	core_length[0]	0x02105	80	core_length[0]
0x06	0x02106	03	flags[0]	0x02106	03	flags[0]
0x07	0x02107	00	flags[1]	0x02107	00	flags[1]
0x08	0x02108	01	protocol_id[0]	0x02108	01	protocol_id[0]
0x09	0x02109	00	protocol_id[1]	0x02109	00	protocol_id[1]
0x0A	0x0210A	05	protocol_version[0]	0x0210A	05	protocol_version[0]
0x0B	0x0210B	01	checksum_type[0]	0x0210B	01	checksum_type[0]
0x0C	0x0210C	3B	target_id[0]	0x0210C	4B	target_id[0]
0x0D	0x0210D	F1	target_id[1]	0x0210D	F2	target_id[1]
0x0E	0x0210E	01	can_mode_mask[0]	0x0210E	01	can_mode_mask[0]
0x0F	0x0210F	00	can_id_format[0]	0x0210F	00	can_id_format[0]
0x10	0x02110	01	app_version[0]	0x02110	02	app_version[0]
0x11	0x02111	06	app_version[1]	0x02111	00	app_version[1]
0x12	0x02112	01	app_version[2]	0x02112	01	app_version[2]
0x13	0x02113	00	app_version[3]	0x02113	00	app_version[3]
0x14	0x02114	99	app_build[0]	0x02114	99	app_build[0]

offset	F13 device address	value	F13欄位[byte]	F24 device address	value	F24欄位[byte]
0x15	0x02115	28	app_build[1]	0x02115	28	app_build[1]
0x16	0x02116	35	app_build[2]	0x02116	35	app_build[2]
0x17	0x02117	01	app_build[3]	0x02117	01	app_build[3]
0x18	0x02118	54	image_size[0]	0x02118	38	image_size[0]
0x19	0x02119	4F	image_size[1]	0x02119	10	image_size[1]
0x1A	0x0211A	00	image_size[2]	0x0211A	00	image_size[2]
0x1B	0x0211B	00	image_size[3]	0x0211B	00	image_size[3]
0x1C	0x0211C	00	app_start[0]	0x0211C	00	app_start[0]
0x1D	0x0211D	20	app_start[1]	0x0211D	20	app_start[1]
0x1E	0x0211E	00	app_start[2]	0x0211E	00	app_start[2]
0x1F	0x0211F	00	app_start[3]	0x0211F	00	app_start[3]

Metadata 0x20..0x3F

offset	F13 device address	value	F13欄位[byte]	F24 device address	value	F24欄位[byte]
0x20	0x02120	00	app_capacity[0]	0x02120	00	app_capacity[0]
0x21	0x02121	E0	app_capacity[1]	0x02121	DC	app_capacity[1]
0x22	0x02122	01	app_capacity[2]	0x02122	03	app_capacity[2]
0x23	0x02123	00	app_capacity[3]	0x02123	00	app_capacity[3]
0x24	0x02124	00	flash_size[0]	0x02124	00	flash_size[0]
0x25	0x02125	00	flash_size[1]	0x02125	00	flash_size[1]
0x26	0x02126	02	flash_size[2]	0x02126	04	flash_size[2]
0x27	0x02127	00	flash_size[3]	0x02127	00	flash_size[3]
0x28	0x02128	00	boot_size[0]	0x02128	00	boot_size[0]
0x29	0x02129	20	boot_size[1]	0x02129	20	boot_size[1]
0x2A	0x0212A	00	boot_size[2]	0x0212A	00	boot_size[2]
0x2B	0x0212B	00	boot_size[3]	0x0212B	00	boot_size[3]
0x2C	0x0212C	00	erase_size[0]	0x0212C	00	erase_size[0]
0x2D	0x0212D	04	erase_size[1]	0x0212D	04	erase_size[1]
0x2E	0x0212E	00	erase_size[2]	0x0212E	00	erase_size[2]
0x2F	0x0212F	00	erase_size[3]	0x0212F	00	erase_size[3]
0x30	0x02130	04	nominal_rate_mask[0]	0x02130	04	nominal_rate_mask[0]
0x31	0x02131	00	nominal_rate_mask[1]	0x02131	00	nominal_rate_mask[1]
0x32	0x02132	00	nominal_rate_mask[2]	0x02132	00	nominal_rate_mask[2]
0x33	0x02133	00	nominal_rate_mask[3]	0x02133	00	nominal_rate_mask[3]
0x34	0x02134	00	data_rate_mask[0]	0x02134	00	data_rate_mask[0]
0x35	0x02135	00	data_rate_mask[1]	0x02135	00	data_rate_mask[1]
0x36	0x02136	00	data_rate_mask[2]	0x02136	00	data_rate_mask[2]
0x37	0x02137	00	data_rate_mask[3]	0x02137	00	data_rate_mask[3]

offset	F13 device address	value	F13欄位[byte]	F24 device address	value	F24欄位[byte]
0x38	0x02138	00	request_id[0]	0x02138	00	request_id[0]
0x39	0x02139	07	request_id[1]	0x02139	07	request_id[1]
0x3A	0x0213A	00	request_id[2]	0x0213A	00	request_id[2]
0x3B	0x0213B	00	request_id[3]	0x0213B	00	request_id[3]
0x3C	0x0213C	01	response_id[0]	0x0213C	01	response_id[0]
0x3D	0x0213D	07	response_id[1]	0x0213D	07	response_id[1]
0x3E	0x0213E	00	response_id[2]	0x0213E	00	response_id[2]
0x3F	0x0213F	00	response_id[3]	0x0213F	00	response_id[3]

Metadata 0x40..0x5F

offset	F13 device address	value	F13欄位[byte]	F24 device address	value	F24欄位[byte]
0x40	0x02140	52	mcu_part ASCII[0]	0x02140	52	mcu_part ASCII[0]
0x41	0x02141	35	mcu_part ASCII[1]	0x02141	37	mcu_part ASCII[1]
0x42	0x02142	46	mcu_part ASCII[2]	0x02142	46	mcu_part ASCII[2]
0x43	0x02143	31	mcu_part ASCII[3]	0x02143	31	mcu_part ASCII[3]
0x44	0x02144	30	mcu_part ASCII[4]	0x02144	32	mcu_part ASCII[4]
0x45	0x02145	42	mcu_part ASCII[5]	0x02145	34	mcu_part ASCII[5]
0x46	0x02146	4D	mcu_part ASCII[6]	0x02146	46	mcu_part ASCII[6]
0x47	0x02147	47	mcu_part ASCII[7]	0x02147	50	mcu_part ASCII[7]
0x48	0x02148	00	mcu_part ASCII[8]	0x02148	4A	mcu_part ASCII[8]
0x49	0x02149	00	mcu_part ASCII[9]	0x02149	35	mcu_part ASCII[9]
0x4A	0x0214A	00	mcu_part ASCII[10]	0x0214A	78	mcu_part ASCII[10]
0x4B	0x0214B	00	mcu_part ASCII[11]	0x0214B	46	mcu_part ASCII[11]
0x4C	0x0214C	00	mcu_part ASCII[12]	0x0214C	42	mcu_part ASCII[12]
0x4D	0x0214D	00	mcu_part ASCII[13]	0x0214D	00	mcu_part ASCII[13]
0x4E	0x0214E	00	mcu_part ASCII[14]	0x0214E	00	mcu_part ASCII[14]
0x4F	0x0214F	00	mcu_part ASCII[15]	0x0214F	00	mcu_part ASCII[15]
0x50	0x02150	52	mcu_family ASCII[0]	0x02150	52	mcu_family ASCII[0]
0x51	0x02151	4C	mcu_family ASCII[1]	0x02151	4C	mcu_family ASCII[1]
0x52	0x02152	37	mcu_family ASCII[2]	0x02152	37	mcu_family ASCII[2]
0x53	0x02153	38	mcu_family ASCII[3]	0x02153	38	mcu_family ASCII[3]
0x54	0x02154	2F	mcu_family ASCII[4]	0x02154	2F	mcu_family ASCII[4]
0x55	0x02155	46	mcu_family ASCII[5]	0x02155	46	mcu_family ASCII[5]
0x56	0x02156	31	mcu_family ASCII[6]	0x02156	32	mcu_family ASCII[6]
0x57	0x02157	33	mcu_family ASCII[7]	0x02157	34	mcu_family ASCII[7]

offset	F13 device address	value	F13欄位[byte]	F24 device address	value	F24欄位[byte]
0x58	0x02158	00	mcu_family ASCII[8]	0x02158	00	mcu_family ASCII[8]
0x59	0x02159	00	mcu_family ASCII[9]	0x02159	00	mcu_family ASCII[9]
0x5A	0x0215A	00	mcu_family ASCII[10]	0x0215A	00	mcu_family ASCII[10]
0x5B	0x0215B	00	mcu_family ASCII[11]	0x0215B	00	mcu_family ASCII[11]
0x5C	0x0215C	04	minimum_size[0]	0x0215C	04	minimum_size[0]
0x5D	0x0215D	02	minimum_size[1]	0x0215D	02	minimum_size[1]
0x5E	0x0215E	00	minimum_size[2]	0x0215E	00	minimum_size[2]
0x5F	0x0215F	00	minimum_size[3]	0x0215F	00	minimum_size[3]

Metadata 0x60..0x7F

offset	F13 device address	value	F13欄位[byte]	F24 device address	value	F24欄位[byte]
0x60	0x02160	55	UPD1 magic[0]	0x02160	55	UPD1 magic[0]
0x61	0x02161	50	UPD1 magic[1]	0x02161	50	UPD1 magic[1]
0x62	0x02162	44	UPD1 magic[2]	0x02162	44	UPD1 magic[2]
0x63	0x02163	31	UPD1 magic[3]	0x02163	31	UPD1 magic[3]
0x64	0x02164	03	policy_version[0]	0x02164	03	policy_version[0]
0x65	0x02165	20	policy_size[0]	0x02165	20	policy_size[0]
0x66	0x02166	01	image_role[0]	0x02166	01	image_role[0]
0x67	0x02167	01	update_transports[0]	0x02167	01	update_transports[0]
0x68	0x02168	46	irq_abi[0]	0x02168	46	irq_abi[0]
0x69	0x02169	31	irq_abi[1]	0x02169	32	irq_abi[1]
0x6A	0x0216A	56	irq_abi[2]	0x0216A	56	irq_abi[2]
0x6B	0x0216B	31	irq_abi[3]	0x0216B	31	irq_abi[3]
0x6C	0x0216C	00	minimum_boot[0]	0x0216C	00	minimum_boot[0]
0x6D	0x0216D	07	minimum_boot[1]	0x0216D	01	minimum_boot[1]
0x6E	0x0216E	01	minimum_boot[2]	0x0216E	01	minimum_boot[2]
0x6F	0x0216F	00	minimum_boot[3]	0x0216F	00	minimum_boot[3]
0x70	0x02170	00	device_uart_baud[0]	0x02170	00	device_uart_baud[0]
0x71	0x02171	00	device_uart_baud[1]	0x02171	00	device_uart_baud[1]
0x72	0x02172	00	device_uart_baud[2]	0x02172	00	device_uart_baud[2]
0x73	0x02173	00	device_uart_baud[3]	0x02173	00	device_uart_baud[3]
0x74	0x02174	00	version_control[0]	0x02174	00	version_control[0]
0x75	0x02175	00	version_control[1]	0x02175	00	version_control[1]
0x76	0x02176	00	version_control[2]	0x02176	00	version_control[2]
0x77	0x02177	00	version_control[3]	0x02177	00	version_control[3]

offset	F13 device address	value	F13欄位[byte]	F24 device address	value	F24欄位[byte]
0x78	0x02178	00	reserved_zero[0]	0x02178	00	reserved_zero[0]
0x79	0x02179	00	reserved_zero[1]	0x02179	00	reserved_zero[1]
0x7A	0x0217A	00	reserved_zero[2]	0x0217A	00	reserved_zero[2]
0x7B	0x0217B	00	reserved_zero[3]	0x0217B	00	reserved_zero[3]
0x7C	0x0217C	00	reserved_zero[4]	0x0217C	00	reserved_zero[4]
0x7D	0x0217D	00	reserved_zero[5]	0x0217D	00	reserved_zero[5]
0x7E	0x0217E	00	reserved_zero[6]	0x0217E	00	reserved_zero[6]
0x7F	0x0217F	00	reserved_zero[7]	0x0217F	00	reserved_zero[7]

Metadata 0x80..0x9F

offset	F13 device address	value	F13欄位[byte]	F24 device address	value	F24欄位[byte]
0x80	0x02180	FF	reserved_FF[0]	0x02180	FF	reserved_FF[0]
0x81	0x02181	FF	reserved_FF[1]	0x02181	FF	reserved_FF[1]
0x82	0x02182	FF	reserved_FF[2]	0x02182	FF	reserved_FF[2]
0x83	0x02183	FF	reserved_FF[3]	0x02183	FF	reserved_FF[3]
0x84	0x02184	FF	reserved_FF[4]	0x02184	FF	reserved_FF[4]
0x85	0x02185	FF	reserved_FF[5]	0x02185	FF	reserved_FF[5]
0x86	0x02186	FF	reserved_FF[6]	0x02186	FF	reserved_FF[6]
0x87	0x02187	FF	reserved_FF[7]	0x02187	FF	reserved_FF[7]
0x88	0x02188	FF	reserved_FF[8]	0x02188	FF	reserved_FF[8]
0x89	0x02189	FF	reserved_FF[9]	0x02189	FF	reserved_FF[9]
0x8A	0x0218A	FF	reserved_FF[10]	0x0218A	FF	reserved_FF[10]
0x8B	0x0218B	FF	reserved_FF[11]	0x0218B	FF	reserved_FF[11]
0x8C	0x0218C	FF	reserved_FF[12]	0x0218C	FF	reserved_FF[12]
0x8D	0x0218D	FF	reserved_FF[13]	0x0218D	FF	reserved_FF[13]
0x8E	0x0218E	FF	reserved_FF[14]	0x0218E	FF	reserved_FF[14]
0x8F	0x0218F	FF	reserved_FF[15]	0x0218F	FF	reserved_FF[15]
0x90	0x02190	FF	reserved_FF[16]	0x02190	FF	reserved_FF[16]
0x91	0x02191	FF	reserved_FF[17]	0x02191	FF	reserved_FF[17]
0x92	0x02192	FF	reserved_FF[18]	0x02192	FF	reserved_FF[18]
0x93	0x02193	FF	reserved_FF[19]	0x02193	FF	reserved_FF[19]
0x94	0x02194	FF	reserved_FF[20]	0x02194	FF	reserved_FF[20]
0x95	0x02195	FF	reserved_FF[21]	0x02195	FF	reserved_FF[21]
0x96	0x02196	FF	reserved_FF[22]	0x02196	FF	reserved_FF[22]
0x97	0x02197	FF	reserved_FF[23]	0x02197	FF	reserved_FF[23]

offset	F13 device address	value	F13欄位[byte]	F24 device address	value	F24欄位[byte]
0x98	0x02198	FF	reserved_FF[24]	0x02198	FF	reserved_FF[24]
0x99	0x02199	FF	reserved_FF[25]	0x02199	FF	reserved_FF[25]
0x9A	0x0219A	FF	reserved_FF[26]	0x0219A	FF	reserved_FF[26]
0x9B	0x0219B	FF	reserved_FF[27]	0x0219B	FF	reserved_FF[27]
0x9C	0x0219C	FF	reserved_FF[28]	0x0219C	FF	reserved_FF[28]
0x9D	0x0219D	FF	reserved_FF[29]	0x0219D	FF	reserved_FF[29]
0x9E	0x0219E	FF	reserved_FF[30]	0x0219E	FF	reserved_FF[30]
0x9F	0x0219F	FF	reserved_FF[31]	0x0219F	FF	reserved_FF[31]

Metadata 0xA0..0xBF

offset	F13 device address	value	F13欄位[byte]	F24 device address	value	F24欄位[byte]
0xA0	0x021A0	FF	reserved_FF[32]	0x021A0	FF	reserved_FF[32]
0xA1	0x021A1	FF	reserved_FF[33]	0x021A1	FF	reserved_FF[33]
0xA2	0x021A2	FF	reserved_FF[34]	0x021A2	FF	reserved_FF[34]
0xA3	0x021A3	FF	reserved_FF[35]	0x021A3	FF	reserved_FF[35]
0xA4	0x021A4	FF	reserved_FF[36]	0x021A4	FF	reserved_FF[36]
0xA5	0x021A5	FF	reserved_FF[37]	0x021A5	FF	reserved_FF[37]
0xA6	0x021A6	FF	reserved_FF[38]	0x021A6	FF	reserved_FF[38]
0xA7	0x021A7	FF	reserved_FF[39]	0x021A7	FF	reserved_FF[39]
0xA8	0x021A8	FF	reserved_FF[40]	0x021A8	FF	reserved_FF[40]
0xA9	0x021A9	FF	reserved_FF[41]	0x021A9	FF	reserved_FF[41]
0xAA	0x021AA	FF	reserved_FF[42]	0x021AA	FF	reserved_FF[42]
0xAB	0x021AB	FF	reserved_FF[43]	0x021AB	FF	reserved_FF[43]
0xAC	0x021AC	FF	reserved_FF[44]	0x021AC	FF	reserved_FF[44]
0xAD	0x021AD	FF	reserved_FF[45]	0x021AD	FF	reserved_FF[45]
0xAE	0x021AE	FF	reserved_FF[46]	0x021AE	FF	reserved_FF[46]
0xAF	0x021AF	FF	reserved_FF[47]	0x021AF	FF	reserved_FF[47]
0xB0	0x021B0	FF	reserved_FF[48]	0x021B0	FF	reserved_FF[48]
0xB1	0x021B1	FF	reserved_FF[49]	0x021B1	FF	reserved_FF[49]
0xB2	0x021B2	FF	reserved_FF[50]	0x021B2	FF	reserved_FF[50]
0xB3	0x021B3	FF	reserved_FF[51]	0x021B3	FF	reserved_FF[51]
0xB4	0x021B4	FF	reserved_FF[52]	0x021B4	FF	reserved_FF[52]
0xB5	0x021B5	FF	reserved_FF[53]	0x021B5	FF	reserved_FF[53]
0xB6	0x021B6	FF	reserved_FF[54]	0x021B6	FF	reserved_FF[54]
0xB7	0x021B7	FF	reserved_FF[55]	0x021B7	FF	reserved_FF[55]

offset	F13 device address	value	F13欄位[byte]	F24 device address	value	F24欄位[byte]
0xB8	0x021B8	FF	reserved_FF[56]	0x021B8	FF	reserved_FF[56]
0xB9	0x021B9	FF	reserved_FF[57]	0x021B9	FF	reserved_FF[57]
0xBA	0x021BA	FF	reserved_FF[58]	0x021BA	FF	reserved_FF[58]
0xBB	0x021BB	FF	reserved_FF[59]	0x021BB	FF	reserved_FF[59]
0xBC	0x021BC	FF	reserved_FF[60]	0x021BC	FF	reserved_FF[60]
0xBD	0x021BD	FF	reserved_FF[61]	0x021BD	FF	reserved_FF[61]
0xBE	0x021BE	FF	reserved_FF[62]	0x021BE	FF	reserved_FF[62]
0xBF	0x021BF	FF	reserved_FF[63]	0x021BF	FF	reserved_FF[63]

Metadata 0xC0..0xDF

offset	F13 device address	value	F13欄位[byte]	F24 device address	value	F24欄位[byte]
0xC0	0x021C0	FF	reserved_FF[64]	0x021C0	FF	reserved_FF[64]
0xC1	0x021C1	FF	reserved_FF[65]	0x021C1	FF	reserved_FF[65]
0xC2	0x021C2	FF	reserved_FF[66]	0x021C2	FF	reserved_FF[66]
0xC3	0x021C3	FF	reserved_FF[67]	0x021C3	FF	reserved_FF[67]
0xC4	0x021C4	FF	reserved_FF[68]	0x021C4	FF	reserved_FF[68]
0xC5	0x021C5	FF	reserved_FF[69]	0x021C5	FF	reserved_FF[69]
0xC6	0x021C6	FF	reserved_FF[70]	0x021C6	FF	reserved_FF[70]
0xC7	0x021C7	FF	reserved_FF[71]	0x021C7	FF	reserved_FF[71]
0xC8	0x021C8	FF	reserved_FF[72]	0x021C8	FF	reserved_FF[72]
0xC9	0x021C9	FF	reserved_FF[73]	0x021C9	FF	reserved_FF[73]
0xCA	0x021CA	FF	reserved_FF[74]	0x021CA	FF	reserved_FF[74]
0xCB	0x021CB	FF	reserved_FF[75]	0x021CB	FF	reserved_FF[75]
0xCC	0x021CC	FF	reserved_FF[76]	0x021CC	FF	reserved_FF[76]
0xCD	0x021CD	FF	reserved_FF[77]	0x021CD	FF	reserved_FF[77]
0xCE	0x021CE	FF	reserved_FF[78]	0x021CE	FF	reserved_FF[78]
0xCF	0x021CF	FF	reserved_FF[79]	0x021CF	FF	reserved_FF[79]
0xD0	0x021D0	FF	reserved_FF[80]	0x021D0	FF	reserved_FF[80]
0xD1	0x021D1	FF	reserved_FF[81]	0x021D1	FF	reserved_FF[81]
0xD2	0x021D2	FF	reserved_FF[82]	0x021D2	FF	reserved_FF[82]
0xD3	0x021D3	FF	reserved_FF[83]	0x021D3	FF	reserved_FF[83]
0xD4	0x021D4	FF	reserved_FF[84]	0x021D4	FF	reserved_FF[84]
0xD5	0x021D5	FF	reserved_FF[85]	0x021D5	FF	reserved_FF[85]
0xD6	0x021D6	FF	reserved_FF[86]	0x021D6	FF	reserved_FF[86]
0xD7	0x021D7	FF	reserved_FF[87]	0x021D7	FF	reserved_FF[87]

offset	F13 device address	value	F13欄位[byte]	F24 device address	value	F24欄位[byte]
0xD8	0x021D8	FF	reserved_FF[88]	0x021D8	FF	reserved_FF[88]
0xD9	0x021D9	FF	reserved_FF[89]	0x021D9	FF	reserved_FF[89]
0xDA	0x021DA	FF	reserved_FF[90]	0x021DA	FF	reserved_FF[90]
0xDB	0x021DB	FF	reserved_FF[91]	0x021DB	FF	reserved_FF[91]
0xDC	0x021DC	FF	reserved_FF[92]	0x021DC	FF	reserved_FF[92]
0xDD	0x021DD	FF	reserved_FF[93]	0x021DD	FF	reserved_FF[93]
0xDE	0x021DE	FF	reserved_FF[94]	0x021DE	FF	reserved_FF[94]
0xDF	0x021DF	FF	reserved_FF[95]	0x021DF	FF	reserved_FF[95]

Metadata 0xE0..0xFF

offset	F13 device address	value	F13欄位[byte]	F24 device address	value	F24欄位[byte]
0xE0	0x021E0	FF	reserved_FF[96]	0x021E0	FF	reserved_FF[96]
0xE1	0x021E1	FF	reserved_FF[97]	0x021E1	FF	reserved_FF[97]
0xE2	0x021E2	FF	reserved_FF[98]	0x021E2	FF	reserved_FF[98]
0xE3	0x021E3	FF	reserved_FF[99]	0x021E3	FF	reserved_FF[99]
0xE4	0x021E4	FF	reserved_FF[100]	0x021E4	FF	reserved_FF[100]
0xE5	0x021E5	FF	reserved_FF[101]	0x021E5	FF	reserved_FF[101]
0xE6	0x021E6	FF	reserved_FF[102]	0x021E6	FF	reserved_FF[102]
0xE7	0x021E7	FF	reserved_FF[103]	0x021E7	FF	reserved_FF[103]
0xE8	0x021E8	FF	reserved_FF[104]	0x021E8	FF	reserved_FF[104]
0xE9	0x021E9	FF	reserved_FF[105]	0x021E9	FF	reserved_FF[105]
0xEA	0x021EA	FF	reserved_FF[106]	0x021EA	FF	reserved_FF[106]
0xEB	0x021EB	FF	reserved_FF[107]	0x021EB	FF	reserved_FF[107]
0xEC	0x021EC	FF	reserved_FF[108]	0x021EC	FF	reserved_FF[108]
0xED	0x021ED	FF	reserved_FF[109]	0x021ED	FF	reserved_FF[109]
0xEE	0x021EE	FF	reserved_FF[110]	0x021EE	FF	reserved_FF[110]
0xEF	0x021EF	FF	reserved_FF[111]	0x021EF	FF	reserved_FF[111]
0xF0	0x021F0	FF	reserved_FF[112]	0x021F0	FF	reserved_FF[112]
0xF1	0x021F1	FF	reserved_FF[113]	0x021F1	FF	reserved_FF[113]
0xF2	0x021F2	FF	reserved_FF[114]	0x021F2	FF	reserved_FF[114]
0xF3	0x021F3	FF	reserved_FF[115]	0x021F3	FF	reserved_FF[115]
0xF4	0x021F4	FF	reserved_FF[116]	0x021F4	FF	reserved_FF[116]
0xF5	0x021F5	FF	reserved_FF[117]	0x021F5	FF	reserved_FF[117]
0xF6	0x021F6	FF	reserved_FF[118]	0x021F6	FF	reserved_FF[118]
0xF7	0x021F7	FF	reserved_FF[119]	0x021F7	FF	reserved_FF[119]

offset	F13 device address	value	F13欄位[byte]	F24 device address	value	F24欄位[byte]
0xF8	0x021F8	FF	reserved_FF[120]	0x021F8	FF	reserved_FF[120]
0xF9	0x021F9	FF	reserved_FF[121]	0x021F9	FF	reserved_FF[121]
0xFA	0x021FA	FF	reserved_FF[122]	0x021FA	FF	reserved_FF[122]
0xFB	0x021FB	FF	reserved_FF[123]	0x021FB	FF	reserved_FF[123]
0xFC	0x021FC	FF	reserved_FF[124]	0x021FC	FF	reserved_FF[124]
0xFD	0x021FD	FF	reserved_FF[125]	0x021FD	FF	reserved_FF[125]
0xFE	0x021FE	FF	reserved_FF[126]	0x021FE	FF	reserved_FF[126]
0xFF	0x021FF	FF	reserved_FF[127]	0x021FF	FF	reserved_FF[127]

本表範例BIN與CRC位置

項目	F13 (MCU CAN device)	F24 (MCU CAN device)
image_size	0x4F54	0x1038
stored image CRC32	0xD30CFB79	0x6ABF121D
CRC32 Flash address	0x06F50..0x06F53	0x03034..0x03037
metadata staging CRC32	0xAAAC5BDC	0x3CDCA2BA

F13 device 範例BIN SHA-256：`8E46743D2A30389E47519EDADB63CEA34A412E9999121B3730EC65175426E633`。 F24 device 範例BIN SHA-256：`4690E533111BEA3F709F5011D739917C07D1F920E0294EB89D63AC857B3CBF4C`。

欄位的語意與修改限制

- core_length=128，metadata 整區為256 bytes；0x80..0xFF 為保留區，依契約全部填FF。
- flags=0x0003、protocol ID / version、checksum type及magic屬於固定契約。 flags 以完整0x0003比對；調整位元定義時同步產生端與驗證端。
- CAN mode mask的bit0代表Classical；目前F13 id_format=0代表standard。nominal_rate_mask的bit2表示500K，mask不是十進位500000。
- target_id / board_id是邏輯識別；MCU part / family是零結尾、其餘零填補ASCII，不是Flash address。
- image_size是實際封裝BIN含最後4-byte CRC的長度；app_capacity是分區允許的最大容量。
- app_version為0x00MMmmpp，app_build為32-bit建置識別。 image_size / CRC由post-build依實際建置結果產生。
- UPD1 image_role：1=DEVICE_APP、2=HOST_BRIDGE_APP；update_transports：bit0 CAN、bit1 I2C、bit2 UART。

- irq_abi是固定向量介面的識別，minimum_boot是最低相容Boot版本，不是App最低版本。
- CAN device 的 policy_version=3，device_uart_baud 為0；`version_control` 位於 `0x74..0x77`，其後 `0x78..0x7F` 為0。 `version_control` 由 App header 產生；`0` 允許任意版本，`1` 由 Boot 在一般 START 限制為同版 / 新版。 操作與維護確認見 Tool 版本政策。

3. 範例程式、產生端與修改位置

用途	原始碼 / 位置	要如何閱讀或修改
F13 metadata生成常數	ftable.inc	此檔現為生成檔；編輯App的app_image_config.h，由App-mode generator結合Boot contract產生常數；不可手改.inc
F13 byte排列 / policy	ftable_RL78_F13.asm	.appmeta從APP_MANIFEST_TABLE開始，.DB4依LE放4 bytes；尾128 bytes填FF
F13實際size / CRC封裝	generate_update_image.ps1	依map決定結尾、patch image_size、計算尾CRC；ExpectedTargetId等驗證亦須與新專案一致
Device metadata 解碼 / 驗證	Boot 原始碼內的 `image_metadata.c/.h`、`image_metadata_config.h`、`image_metadata_profile.c`	ImageMetadataRead32 解碼 LE32；ImageMetadataValidate 對照 g_image_metadata_contract；完整 API 見 C 模組移植
CAN metadata 暫存 / SEAL	can_boot_metadata.inc	MetadataStage 核對順序 / 重送；MetadataSeal 呼叫共用驗證並核對 staging CRC，不在 parser 內擦寫 Flash
F13 CAN解碼與回覆	can_boot_transport.c / can_boot_block.inc	CAN_BootReceive、CAN_BootSend、BlockBegin / Segment / Check
F24 device 契約與封裝	`image_metadata_config.h`、`can_boot_config.h`、`tools/device_common.ps1`、`tools/package_images.ps1`；工程入口	Get-DeviceMetadata 結合映像 / CAN 設定產生欄位，Get-PackagedDeviceApp 核對連結內容與 CRC
共用政策常數	image_policy.h	ROLE / CAN / I2C / UART bits、IRQ ABI、core / policy長度
Tool image解析	firmware_update_profile.cpp	檢查BIN、metadata、CRC及目的地
Tool CAN frame編碼	f13_boot_profile.cpp	BuildStartRequest、BuildBlockRequest、Client::StageMetadata / SendBlock，與下方逐byte表核對

F13組語常數與實際byte的關係

```
; Values from the paired F13 CAN example, not a new device configuration.
IMAGE_MANIFEST_FORMAT_INFO .EQU 0x00038006
IMAGE_MANIFEST_PROTOCOL    .EQU 0x01050001
IMAGE_MANIFEST_TARGET      .EQU 0x0001F13B
; .DB4 emits little-endian words:
; metadata 04..07 = 06 80 03 00
; metadata 08..0B = 01 00 05 01
; metadata 0C..0F = 3B F1 01 00
```

例如改 target_id 要核對 Boot metadata header 的 IMAGE_TARGET_ID、App 的 IMAGE_MANIFEST_TARGET、 post-build驗證及Tool live INFO，不是只修改0x0210C。改CAN ID還須保持control / response與raw720..727不重疊。

C90示例：讀metadata而不依賴struct padding

```
/* The input is a RAM copy beginning at metadata offset 0x00. */
static unsigned long ReadLe32(const unsigned char *p)
{
    return (unsigned long)p[0] |
           ((unsigned long)p[1] << 8) |
           ((unsigned long)p[2] << 16) |
           ((unsigned long)p[3] << 24);
}

static unsigned long ExampleDeviceAppStart(const unsigned char *metadata)
{
    return ReadLe32(metadata + 0x1C);
}
```

此示例示意 LE 及欄位 offset；完整驗證另核對 magic、format、長度與 CRC。 讀RL78遠端Flash須依實際編譯器的far pointer / 讀取API搬到RAM，不能將上例near pointer直接指向所有Flash。 Flash erase / write必須走該MCU的FSL / RFD及保護流程，不能用普通pointer賦值取代。

4. CAN payload逐byte格式

Byte0..Byte7指CAN Data payload，不含ID、DLC、CAN硬體CRC / ACK。不是每個command都DLC8。 下表的「-」代表沒有傳送該byte，不是要求補00。所有offset / size / CRC數值皆LE。

CAN Upgrade：control request（ID0x700）

Command	DLC	B0	B1	B2	B3	B4	B5	B6	B7
HELLO	1	01	-	-	-	-	-	-	-
START	8	02	protocol	target0	target1	size0	size1	size2	size3
FORCE_START	8	0E	protocol	target0	target1	size0	size1	size2	size3
END	1	04	-	-	-	-	-	-	-
ABORT	1	05	-	-	-	-	-	-	-
ENTER	1	06	-	-	-	-	-	-	-
INFO	2	07	page	-	-	-	-	-	-
METADATA	8	08	metaOff0	metaOff1	00	data0	data1	data2	data3
SEAL	8	09	00	00	00	metaCrc0	metaCrc1	metaCrc2	metaCrc3
METADATA_RESET	1	0A	-	-	-	-	-	-	-
BLOCK_BEGIN	8	0B	imageOff0	imageOff1	imageOff2	blockCrc0	blockCrc1	blockCrc2	blockCrc3
BLOCK_COMMIT	8	0C	imageOff0	imageOff1	imageOff2	blockCrc0	blockCrc1	blockCrc2	blockCrc3

protocol目前=05；target目前=0xF13B，所以START B2 / B3=3B / F1。 App只處理INFO / ENTER；HELLO與更新命令供Boot使用，已在Boot時不需再送ENTER。 本表ID為目前F13範例；Tool仍須依metadata / live INFO核對。raw data base ID與block size由INFO page0x1A宣告， 不在metadata的request_id / response_id欄位內；移植時三組ID不得重疊。 size為整個BIN長度、不是capacity。metaOff相對256-byte metadata；imageOff相對整個BIN、64-byte對齊。 data0..3是本次METADATA chunk的4個原始byte；metaCrc是完整256-byte CRC，blockCrc是本區塊實際長度CRC。 BEGIN與COMMIT必須使用相同imageOff / blockCrc；BEGIN不回ACK，COMMIT驗證並寫入成功才回ACK。

Raw image frame（ID0x720..0x727）

Byte	意義
B0	BIN[imageOff + segment*8 + 0]
B1	BIN[imageOff + segment*8 + 1]
B2	BIN[imageOff + segment*8 + 2]
B3	BIN[imageOff + segment*8 + 3]
B4	BIN[imageOff + segment*8 + 4]
B5	BIN[imageOff + segment*8 + 5]
B6	BIN[imageOff + segment*8 + 6]
B7	BIN[imageOff + segment*8 + 7]

segment=CAN_ID-0x720，範圍0..7。這8個byte全部是image，不另放command / sequence / CRC8。 最後不足8但仍4-byte對齊時DLC4，只傳B0..B3；未使用byte不寫Flash。raw frame本身不逐筆回覆。

Response（ID0x701）

Response	DLC	B0	B1	B2	B3	B4	B5	B6	B7
HELLO	8	01	status	protocol	format	target0	target1	capKiB0	capKiB1
INFO	8	07	status	page	state	value0	value1	value2	value3
ENTER	2	06	status	-	-	-	-	-	-
common ACK / NACK	8	echoedCmd	status	detail0	detail1	value0	value1	value2	value3

status=00為成功，非零必須依command / 狀態解讀；例如01 BAD_COMMAND、03 BAD_SEQUENCE、 05 FLASH_ERROR、06 CRC_ERROR、09 BAD_IMAGE。INFO state=1 App、2 Boot；INFO page0的value包含target / protocol / format， 其他page與定義見CAN協定。

ACK command	detail（B2..3）	value（B4..7）
START	0	App起點絕對Flash address
METADATA_RESET	0	256
METADATA	下一個metadata offset	256

ACK command	detail (B2..3)	value (B4..7)
SEAL	256	256
BLOCK_COMMIT	(imageOff / 64)低16 bits	下一個預期Flash絕對address
END	(image_size / 4)低16 bits	device計算的完整image CRC32
ABORT	當前expected image byte offset / 4低16 bits	當前Flash address；不等於可斷線續傳承諾

只讀INFO範例：TX ID0x700 / DLC2 / `07 00`；Boot正常回ID0x701 / DLC8 / `07 00 00 02 3B F1 05 06`，分別是command、OK、page0、Boot state、target LE16、protocol、format。 第一個64-byte block成功的ACK為`0C 00 00 00 40 20 00 00`，最後四byte解為下一個Flash address 0x02040。 ACK範例不是完整燒錄流程；必須先完成live比對、metadata staging / SEAL及START，不能直接拿任意raw frame寫入。

與Custom Transfer / Bridge Self Test的區別

Byte	Transfer request 0x710	Transfer response 0x711	Self Test 0x123
B0	A5 magic	5A magic	11
B1	protocol 01	protocol 01	22
B2	非零sequence	對應sequence	33
B3	command	對應command；錯誤為7F	44
B4	argument0	response0	55
B5	argument1	response1	66
B6	argument2	response2	77
B7	CRC8(Byte0..6)	CRC8(Byte0..6)	counter，每次Start從00遞增、FF後回00

Transfer固定DLC8，CRC8 polynomial07、init00、xorout00、不反射；命令參數見 Transfer協定。 Self Test不是更新協定，也沒有Byte7 CRC；需CAN analyzer觀察實際bus訊號。

5. 改成自己的專案：修改順序與驗證

1. 先選定 MCU CAN device 工程，確認硬體手冊的 Flash / RAM 範圍、CAN clock 與實體向量位址。
2. 在 Boot 的 `image_metadata_config.h` 設定分區、target ID 與 IRQ ABI； `can_boot_config.h` 設定 CAN IDs / rate，`boot_app_handoff.h` 設定共用 RAM 交接。 Linker 的 App 起點及向量表 / metadata 配置對應同一分區。
3. 在 App 編輯 `app_image_config.h` 與 `app_irq_handlers.h`。 F13 由 PowerShell 產生 `ftable.inc`；F24 由 `Get-DeviceMetadata` 產生 metadata INC，兩者皆由 App-local pre-build 產表。

4. 建置後核對 map、封裝 BIN 的 metadata / CRC 及所有 BR 目的地。 App 起點是 link 設定；Tool 的 App Start Override 用於比對，不會重定位 BIN。
5. 在 Tool 載入 BIN，先做離線驗證，再以 Discover 取得 live device 資料。 更新流程將完整 metadata staging / SEAL 後，才允許進入 START 與區塊寫入。
6. 封裝欄位或契約變動時，一併更新產生器、Boot 驗證、Tool parser 與測試。

表格來源

逐 byte 範例取自各 device 工程的實際 update BIN。 位址由 App 起點及欄位 offset 計算；App 長度與 CRC 依每次建置結果產生。